

FinTrack – Finance Tracker Website

A PROJECT REPORT for Major Project-II (CA401P) Session (2025-26)

Submitted by

**Paras Chandrawanshi
(202410116100140)**

**Nitin Gangwar
(202410116100136)**

**Neha Agnihotri
(202410116100131)**

**Rishabh Singh Bhadauriya
(202410116100166)**

**Submitted in partial fulfilment of the
Requirements for the Degree of**

MASTER OF COMPUTER APPLICATION

Under the Supervision of

**Dr. Amit Kumar
(Assistant Professor)**

**Mr. Shish Pal Jatav
(Assistant Professor)**



Submitted to

**Department Of computer Applications
Kiet Group of Institutions, Ghaziabad
Uttar Pradesh -20120
(June-2026)**

CERTIFICATE

Certified that **Paras Chandrawanshi (202410116100140)**, **Nitin Gangwar (202410116100136)**, **Neha Agnihotri (202410116100131)**, and **Rishabh Singh Bhadauriya (202410116100166)** have carried out the project work entitled **“FinTrack – Finance Tracker Website”** (AI-Based Project) for **Master of Computer Application** from KIET Group of Institutions (an Autonomous College) under Dr. A.P.J. Abdul Kalam Technical University, Lucknow under our supervision. The project report embodies original work, and studies are carried out by the students themselves. The contents of the project report do not form the basis for the award of any other degree to the candidates or to anybody else from this or any other University/Institution.

Paras Chandrawanshi (202410116100140)

Nitin Gangwar (202410116100136)

Neha Agnihotri (202410116100131)

Rishabh Singh Bhadauriya(202410116100166)

This is to certify that the above statement made by the candidate is correct to the best of my knowledge.

Date:

Dr. Amit Kumar
Assistant Professor
Department of Computer
Applications
KIET Group of
Institutions, Ghaziabad

Mr. Shish Pal Jatav
Assistant Professor
Department of Computer
Applications
KIET Group of
Institutions, Ghaziabad

Dr. Sachin Malhotra
Dean
Department of Computer
Applications
KIET Group of
Institutions, Ghaziabad

ABSTRACT

The project titled “**FinTrack – AI-Based Finance Tracker**” is a comprehensive financial management application developed to help users efficiently monitor, analyze, and manage their personal expenses and savings through intelligent automation and data analysis. Traditional methods of expense tracking are often time-consuming, inaccurate, and lack proper financial insights. This project addresses these challenges by providing a smart and user-friendly platform that enables users to record transactions, track spending patterns, and make better financial decisions.

The system is developed using **Python** along with modern libraries and frameworks for backend processing, data handling, and AI-based analysis. The application incorporates **Artificial Intelligence and Machine Learning techniques** to analyze user spending behavior, categorize expenses automatically, and provide smart financial recommendations and budget predictions. The platform also supports data visualization features such as charts and graphs, allowing users to clearly understand their financial activities and monthly expenditure trends.

The application includes multiple modules such as user authentication, expense management, income tracking, budget planning, report generation, and AI-powered analytics. It also provides secure data storage and real-time financial insights, making it an intelligent and efficient finance management solution for students, professionals, and small businesses.

Keywords: Artificial Intelligence, Finance Tracker, Python, Machine Learning, Expense Management, Budget Prediction, Data Visualization, Smart Financial Analysis.

ACKNOWLEDGEMENTS

Success in life is never achieved alone. I express my deepest gratitude to my thesis supervisor, Dr. Amit Kumar (Associate Professor) and Sheesh Pal Jatav (Assistant professor) for her guidance, assistance, and encouragement throughout my project work. Her insightful ideas, comments, and suggestions have been invaluable in helping me complete this project successfully.

I am also immensely grateful to Dr. Sachin Malhotra, Professor and Head of, the Department of Computer Applications, for his valuable feedback and administrative support on several occasions. I am fortunate to have many supportive friends who have been of great help during critical moments of this journey.

Lastly, I would like to extend my heartfelt thanks to my family members and all those who have provided me with moral support and assistance, both directly and indirectly. Completing this work on time would not have been possible without their unwavering support. Their constant love and encouragement have filled my life with joy and happiness.

Paras Chandrawanshi

Nitin Gangwar

Neha Agnihotri

Rishabh Singh Bhadauriya

TABLE OF CONTENTS

S.no.	Title	Page No.
	Certificate	ii
	Abstract	iii
	Acknowledgment	iv
	Table of Contents	v–vi
	CHAPTER 1: Introduction	7–10
1.1	Background	7
1.2	Problem Review	7
1.3	Objectives	8
1.4	Key Features	9
1.5	Scope	9
1.6	Hardware & Software Requirements	9
1.7	Background Study	10
	CHAPTER 2: Feasibility Study	11–13
2.1	Economic Feasibility	11
2.2	Technical Feasibility	11
2.3	Operational Feasibility	12
2.4	Behavioral Feasibility	13
	CHAPTER 3: Software Requirement Specification (SRS)	14–22
3.1	Functionalities	14 - 16
3.2	User and Characteristics	17
3.3	Features of Project	18 -19
3.4	Features of Admin	20
3.5	Features of User	21 - 22
	CHAPTER 4: Hardware and Software Requirements	23

	CHAPTER 5: Project Flow	24 -25
	CHAPTER 6: System Requirements	26–30
6.1	Functional Requirements	26–27
6.2	Non-Functional Requirements	28–29
6.3	Design Goals	30
	CHAPTER 7: System Design	31–35
7.1	Primary Design Phase	31–32
7.2	Secondary Design Phase	33–34
7.3	User Interface Design	34–35
	CHAPTER 8: Project Outcomes	36–43
8.1	User Module	36–39
8.2	Admin Module	40–42
8.3	Database Module	43
	CHAPTER 9: Testing of Car Rental Project	44–49
9.1	Introduction to Testing	44
9.2	Types of Testing	45
9.3	Test Plan and Execution	46
9.4	Results and Test Reports	47
9.5	Challenges Faced	48–49
	CHAPTER 10- Proposed Time Duration	50–51
	CONCLUSION	52–54
	REFERENCES	55–56

CHAPTER – 1

INTRODUCTION

1.1 Background

The financial management sector has experienced rapid growth due to increasing digitalization, online transactions, and the need for efficient expense tracking systems. Modern users prefer smart financial applications that help them manage income, expenses, savings, and financial goals in a simple and organized way. A web-based financial tracking system simplifies personal finance management by allowing users to monitor transactions, analyze spending patterns, and maintain financial records from a single platform.

The finance industry has undergone significant transformation in recent years, increasing the demand for intelligent and user-friendly financial solutions. Traditional methods of managing finances, such as manual bookkeeping and spreadsheet maintenance, often suffer from inefficiencies including lack of real-time tracking, poor financial analysis, limited accessibility, and difficulty in maintaining records accurately. These limitations reduce financial awareness and make effective money management challenging for users.

1.2 Problem Review

Traditional financial management systems involve manual calculations, paper-based records, and inefficient methods of tracking expenses and income. Users often face difficulties in monitoring spending habits, managing budgets, and analyzing financial data without proper digital support. Additionally, maintaining records manually increases the chances of errors, data loss, and inaccurate financial reporting.

Thus, there is a need for a reliable, automated, user-friendly digital system that handles end-to-end manage finance efficiently.

To address these challenges, the **FinTrack – AI Powered Expense Tracker** has been conceptualized and developed as a Python-based web application. This system provides users with a seamless and interactive platform to manage their financial activities efficiently. With features such as AI-based expense analysis, real-time transaction tracking, secure authentication, and detailed financial insights, the application ensures smooth end-to-end financial management for users.

The project is designed using modern technologies—Python with Flask/Django for backend development, HTML, CSS, and JavaScript for frontend design, and MySQL/SQLite for efficient database management. By adopting these technologies, the system ensures scalability, security, reliability, and high performance.

1.4 Objective

The main objectives of this project are:

- To develop a web-based platform for expense tracking and financial management.
- To integrate an AI-based smart expense analysis and recommendation system.
- To enable users to record, manage, and monitor their income and expenses efficiently.
- To provide admin and user dashboards for managing financial records and transaction activities.
- To build a scalable and secure system using Python-based technologies.

From an organizational perspective, the system automates repetitive manual financial tasks, thereby reducing the workload for users and businesses. Users can easily manage daily transactions, monitor spending patterns in real-time, and access detailed financial reports and summaries. This leads to improved financial planning, reduced calculation errors, and better money management.

Beyond solving present-day challenges, the project also envisions future expansions such as AI-driven budgeting assistance, predictive financial analytics, cloud synchronization, and dedicated mobile applications. Thus, this project represents a step toward digital transformation in financial management systems, aiming to create a modern, user-centric, and future-ready platform that benefits individuals and organizations alike.

1.5 Key Features

- User Registration & Login
- Expense and Income Management
- AI-Based Smart Expense Analysis and Recommendations
- Real-Time Transaction Tracking
- Financial Dashboard with Reports and Insights
- Transaction Update / Delete Management
- Secure Database Management
- Responsive User Interface

1.6 Scope

The project is designed for educational, commercial, and enterprise use. It supports:

- Personal finance and expense management
- Small businesses and startups for financial tracking
- Budget planning and spending analysis
- College project demonstration
- Scalable future expansion such as multi-user and cloud-based financial management support

1.7 Hardware & Software Requirements

Hardware

- Laptop/PC
- Minimum 4GB RAM
- Stable Internet Connection

Software

- Python
- Flask / Django
- Python libraries and Streamlit Components
- MySQL / SQLite
- VS Code / PyCharm
- Chrome / Firefox Browser
- GitHub (for version control)

1.9 Background

The rapid growth of digital financial services and expense management applications inspired the development of this application. Increasing awareness about personal finance management and smart budgeting has created a demand for efficient and user-friendly financial tracking systems. This project follows Agile methodology and uses modern Python-based web technologies to ensure fast performance, high reliability, security, scalability, and easy maintainability.

CHAPTER – 2

FEASIBILITY STUDY

A Feasibility Study is conducted to evaluate whether a project is practical, viable, and cost-effective before full-scale development. It analyzes the various factors affecting the success of the proposed system, including economic, technical, operational, and behavioral aspects.

For the **FinTrack – AI Powered Expense Tracker**, the feasibility study confirms that the system can be successfully developed using the chosen technologies and can operate efficiently in real-world financial management scenarios.

2.1 Economic Feasibility

Economic Feasibility examines whether the project is financially viable and whether the benefits outweigh the associated costs.

In this project:

- No significant financial investment is required for development since all major technologies used are open-source.
- Development does not require commercial tools or expensive software licenses.
- Hosting can be done on low-cost or free-tier platforms such as Netlify (Frontend) and Render / Cyclic / Railway (Backend).
- MongoDB Atlas offers a free shared cluster, which is sufficient for academic and small-scale deployment.
- Hardware costs are minimal, requiring only a basic laptop or desktop with internet access.

- The system reduces manual administrative efforts by automating course management, enrollment, and user tracking, thereby lowering operational costs in real-world educational scenarios.

2.2 Technical Feasibility

Technical Feasibility determines whether the required technologies, development platforms, and tools are available and adequate for the successful implementation of the system.

For this project:

- Python with Flask/Django provides a powerful and flexible backend framework suitable for building secure and scalable financial management applications.
- HTML, CSS, and JavaScript provide responsive and interactive frontend support for creating a user-friendly interface.
- MySQL/SQLite provides a reliable and efficient database solution for storing user records, transaction details, budgets, and financial reports.
- All technologies used are modern, stable, well-documented, and supported by large developer communities.
- The system architecture is highly scalable and can accommodate future enhancements like AI-based financial recommendations, predictive analytics, cloud synchronization, and mobile application support.
- No specialized hardware is required; a regular development system with VS Code and Node.js installed is sufficient.

2.3 Operational Feasibility

Operational Feasibility evaluates whether the proposed system can function effectively in real operational environment and whether users can adapt to it easily.

In this project:

- The system has a clean and intuitive interface, making it simple for users to record expenses, manage income, and analyze financial data.

- User operations such as adding transactions, updating records, viewing reports, and tracking expenses are straightforward and require minimal training.
- The system significantly reduces manual workload for individuals and businesses, improving financial accuracy and efficiency.
- Real-time data updates ensure users always have access to the latest transaction records and financial summaries without delays.
- The system supports responsive design, making it accessible across desktops, tablets, and mobile devices.

2.4 Behavioral Feasibility

, Behavioral Feasibility assesses how users will accept and adapt to the new system.

For this project:

- Users are already familiar with digital finance applications and online banking systems, so they can easily understand the expense tracking and budgeting process.
- Minimal learning is required to navigate the system because of its simple interface, form-based inputs, and clear instructions.
- The system promotes transparency by allowing users to track expenses, monitor savings, and analyze spending habits effectively.
- Users find the centralized dashboard convenient because all financial activities and reports are available in one place.
- User feedback can be easily integrated in future versions to improve system performance, financial analysis features, and overall user experience.

CHAPTER – 3

SOFTWARE REQUIREMENTS SPECIFICATION

The Software Requirements Specification (SRS) describes all functional, non-functional, user-based, and system-level requirements necessary for the successful development of the **FinTrack – AI Powered Finance Tracker**. The purpose of this chapter is to ensure that every requirement is clearly defined, validated, and understood before development and testing.

SRS acts as a communication bridge between developers, users, administrators, and stakeholders.

3.1 Functionalities

The core functionalities of the FinTrack – AI Powered Finance Tracker are as follows:

1. User Registration & Authentication

- New users can create an account using name, email, phone number, and password.
- Passwords are stored securely using hashing/encryption techniques.
- Existing users can log in to access expense tracking, financial reports, and budgeting features.
- Admin has a separate login panel for managing users, transactions, and system activities.

2. Expense & Income Management

- a. Users can add daily expenses and income records.
- b. Users can categorize transactions such as:
 - Food
 - Shopping
 - Travel
 - Bills
 - Salary

- Entertainment
- c. Users can update or delete transaction records whenever required.
- d. The system automatically calculates total income, expenses, and remaining balance

3. Smart Expense Analysis & Search

- Users can search transaction history based on category, amount, or date.
- AI-based analysis helps users understand spending patterns and financial habits.
- Filters help narrow down transaction searches using:
 - Transaction Category
 - Date Range
 - Expense/Income Type
 - Amount Range

4. Financial Dashboard

- a. Users can view detailed financial summaries including:
 - Total Income
 - Total Expenses
 - Current Balance
 - Monthly Spending
 - Category-wise Expense Analysis
- b. Dashboard displays charts and graphical reports for better understanding of financial activities.

5. Budget Management

- Users can set monthly or weekly budgets for different categories.
- The system provides alerts when spending exceeds the budget limit.
- Budget details are stored securely in the database.

6. Admin Dashboard

Admin has complete control of the system, including:

- View total users, total transactions, and financial activities.
- Manage user accounts and transaction records.
- Monitor system usage and reports.
- Delete invalid or unwanted records.
- Generate financial summaries and analytics.

7. Real-Time Data Updates

- Whenever a transaction is added, updated, or deleted, updated information is displayed instantly.
- The system ensures smooth and responsive user interaction without unnecessary page refreshes.

3.2 User and Characteristics

1. End Users

These are individuals who want to manage personal or business finances effectively.

Characteristics:

- Basic knowledge of digital financial applications.
- Internet-enabled device (mobile/laptop).
- Ability to manage transactions and analyze spending records.
- Expect a fast, simple, and user-friendly interface.

2. Administrator

The admin manages:

- Users
- Transactions
- Financial reports
- System activities

Characteristics:

- Must have login credentials.
- Intermediate computer knowledge.
- Responsible for maintaining and monitoring the system.

3. System Requirements for Users

- A modern browser (Chrome/Firefox).
- Stable internet connection.
- Ability to view dynamic charts and financial reports.

3.3 Features of Project

This project provides the following major features:

1. Expense & Income Tracking

Users can easily add, manage, and monitor income and expense records.

2. AI-Based Smart Financial Recommendations

The system provides intelligent financial suggestions based on user spending patterns and transaction history.

3. Secure Login System

- Password hashing
- Role-based authentication
- Error messages for invalid credentials

4. Detailed Financial Dashboard

Each user dashboard contains:

- Financial summaries
- Expense charts and reports
- Monthly analysis
- Budget tracking details

5. Transparent Transaction Management

Users can view:

- Transaction date
- Transaction category
- Amount details
- Payment type
- Current balance status

6. Easy Transaction Management

Users can update or delete transactions instantly, and the changes are reflected immediately in the database.

7. Admin Reporting

Admin gets summaries of:

- Total users
- Total transactions
- Expense statistics
- System activity reports

8. Database Integration

All data is stored securely in MySQL/SQLite database, ensuring:

- Scalability
- Data security
- Efficient data management

3.4 Features of Admin

Admin has full authority and advanced controls such as:

1. User Management

- View user details and activities.
- Manage or remove user accounts if required.
- Monitor active users and transaction records.

2. Transaction Management

- View all financial transactions in the system.
- Delete invalid transaction records.
- Monitor financial activities and reports.

3. Dashboard Monitoring

- Admin dashboard shows:
- Total users on platform
- Total transactions
- Daily financial activities
- Monthly analytics and reports

4. Report Generation

- Generate financial summaries
- Analyze user spending trends
- Monitor overall system performance

3.5 Features of User

Users enjoy the following functionalities:

1. Account Creation & Login

Allows secure login and profile creation.

2. Expense & Income Tracking

Users can add and manage multiple financial transactions with clear details and categories.

3. Transaction Search Filters

Users can search based on:

- Date
- Category
- Amount
- Transaction type

4. Instant Financial Updates

Users can instantly view updated balances and financial summaries after every transaction.

5. Budget Tracking

Users can set budgets and monitor spending limits effectively.

6. Financial Reports Section

Users can track:

- Current balance
- Monthly expenses
- Savings records
- Expense analysis reports

7. Mobile-Friendly Interface

Responsive UI works smoothly on:

- Android
- iPhone
- Tablets

CHAPTER – 4

HARDWARE & SOFTWARE REQUIREMENTS

Hardware Requirements

- **Processor:** Intel Core i5 or higher (Quad-Core recommended)
- **RAM:** Minimum 8 GB (16 GB recommended for smoother development)
- **Storage:** 500 GB HDD / 256 GB SSD or higher
- **Display:** 15-inch monitor with minimum resolution of 1366×768
- **Network:** Stable internet connection for hosting and deployment

Software Requirements

- **Operating System:** Windows / Linux / macOS
- **Frontend Technology:** Streamlit for building interactive and responsive user interfaces
- **Backend Technology:** Python with Flask / Django for server-side development
- **Database:** MySQL / SQLite for secure and efficient data storage
- **Web Browser:** Google Chrome / Mozilla Firefox (latest version)
- **Code Editor / IDE:** Visual Studio Code (VS Code)
- **Version Control:** Git and GitHub for collaboration and source code management

CHAPTER – 5

PROJECT FLOW

The development of the **FinTrack – AI Powered Expense Tracker** follows a structured Software Development Life Cycle (SDLC) approach. The key phases are:

1. Analysis & Planning

- Identify the problems and limitations of traditional financial management systems.
- Gather requirements from end-users for expense tracking, budgeting, and financial analysis.
- Define the project scope, features, and technology stack.
- Prepare a development plan with timelines and responsibilities.

2. Frontend Development

- Design and implement the user interface using HTML, CSS, and JavaScript.
- Build reusable modules for login, dashboard, expense management, budgeting, and financial reports.
- Ensure responsiveness across multiple devices (desktop, tablet, and mobile).

3. Backend Development

- Set up server-side operations using Python with Flask/Django.
- Create APIs and backend logic to handle user authentication, expense management, budgeting, and financial analysis.
- Implement proper error handling and data security mechanisms.

4. Database Integration

- Design and integrate MySQL/SQLite database tables for storing user details, transaction records, budgets, and financial reports.

- Ensure proper relationships and optimized queries for faster data retrieval.
- Enable real-time updates for transactions and financial summaries.

5. Testing & Debugging

- Conduct unit testing for individual modules and integration testing for complete workflows.
- Perform functional, usability, and security testing.
- Debug errors and refine the system for better stability and reliability.

6. Maintenance & Updates

- Monitor system performance after deployment.
- Fix bugs, optimize performance, and release updates regularly.
- Add future enhancements such as AI-based financial recommendations, synchronization, predictive analytics, and mobile application support according to future requirements.

CHAPTER – 6

SYSTEM REQUIREMENT

System requirements define the essential conditions, functionalities, constraints, and quality standards needed for the successful development and execution of the **FinTrack – AI Powered Expense Tracker**. This chapter outlines both functional and non-functional requirements and the design goals that guide the structural and technical planning of the system.

6.1 Functional Requirements

Functional requirements describe what the system should do. These are directly linked to the core operations of the **FinTrack – AI Powered Expense Tracker**.

1. User Registration & Authentication

1. The system must allow new users to create an account.
2. The user must provide valid email, phone number, name, and secure password.
3. Login should validate registered credentials and allow access to the dashboard.
4. Passwords must be stored in a hashed format.

2. User Login & Dashboard

1. Users must be able to log in securely.
2. After login, users should see their dashboard displaying:
 - Total income
 - Total expenses
 - Current balance
 - Recent transactions
 - Budget summaries

3. Expense & Income Management

1. Users must be able to add income and expense records securely.
2. Transaction entries should allow categories such as:
 - Food
 - Travel
 - Shopping
 - Bills
 - Entertainment
 - Salary
3. Users should be able to edit or delete transactions.
4. Transactions must display essential details like category, amount, date, and transaction type.

4. Financial Dashboard

Clicking dashboard sections must display complete financial details:

- Total income
- Total expenses
- Current balance
- Monthly reports
- Expense analysis charts
- Budget tracking information

5. Real-Time Financial Updates

- The system must ensure transactions are updated instantly.
- Duplicate or invalid transactions must be restricted automatically.
- Financial summaries and balances must update immediately after transaction changes.

6. Budget Management Module

- Users must be able to set monthly or weekly budgets.
- System must calculate remaining budget automatically.
- Budget records must be stored securely in the database.
- System should provide alerts when spending exceeds the limit.

7. Transaction Deletion & Modification

- Users must be able to update or delete transactions.
- Database must update transaction records immediately.
- UI should reflect updated financial data instantly.

8. Admin Module

Admin must be able to:

a. Manage Users

View all registered users and their activities.

b. Manage Transactions

Monitor, edit, or remove invalid transaction records.

c. Manage Reports

Generate financial reports and summaries.

d. Admin Dashboard

Summary of:

- Total Users
- Total Transactions
- Total Expenses
- Active Users

9. Report Generation

Admin and users should be able to view summarized data:

- Monthly expense reports
- Income summaries
- Spending category analysis
- Budget statistics

10. Database Interaction (MySQL / SQLite)

All information must be stored securely in database tables:

- Users
- Transactions
- Budgets

CRUD operations should be efficient and validated.

6.2 Non-Functional Requirements

These define how the system should behave, focusing on performance, security, and user experience.

1. Performance Requirements

- System should respond to user actions within 1–3 seconds.
- Transaction operations and financial reports must load without lag.
- Backend services must be optimized to handle multiple users efficiently.

2. Security Requirements

- Passwords must be hashed securely.
- Only admin should access admin panel.
- All input fields must be validated properly.
- Secure communication should exist between frontend and backend.

3. Usability Requirements

- Clean UI with simple navigation.
- Minimal steps for adding and managing transactions.
- Buttons, forms, and reports must be clearly labeled.

4. Reliability Requirements

- System should work consistently without failure.
- Financial calculations must always remain accurate.
- Data retrieved from the database must always be correct and updated.

5. Availability Requirements

- System should remain accessible whenever required.
- Database system should ensure reliable data availability.

6. Scalability Requirements

- a. The system should support:

- More users
- More transaction records
- More financial reports

b. Database architecture should support future scaling easily.

7. Portability Requirements

a. Should work on all modern browsers:

- a. Chrome
- b. Firefox
- c. Edge
- d. Brave

b. UI should adapt to desktops, laptops, tablets, and mobile devices.

8. Maintainability Requirements

- Code must be modular with separate frontend and backend components.
- Backend APIs and modules must be documented for future changes.
- Database structure must support future modifications easily.

9. Compatibility Requirements

- Frontend should be compatible across devices.
- Backend APIs should support integration with future applications or mobile platforms.

10. Responsiveness Requirements

- UI must adjust according to screen size.
- Pages must remain readable on mobile devices.
- Navigation menus should adapt properly for smaller screens.

6.3 Design Goals

1. Performance Design Goal

- Ensure fast data retrieval from the database.
- Optimize backend operations and queries.
- Maintain efficient processing of financial calculations and reports.

2. Security Design Goal

- Enforce strong authentication and authorization.
- Secure admin-only functionalities.
- Validate all user inputs on both frontend and backend.

3. UI/UX Design Goal

- Provide a visually simple, modern, and responsive interface.
- Ensure users can manage transactions with minimal clicks.
- Maintain a clean and consistent design theme.

4. Maintainability Design Goal

- Modular frontend and backend structure.
- Clean routing and database handling.
- Reusable backend functions and controllers.
- Centralized error handling and validation system.

CHAPTER – 7

SYSTEM DESIGN

System Design is a crucial phase that transforms the gathered requirements into a structured plan for building the **FinTrack – AI Powered Expense Tracker**. It describes how the system's components are arranged, how modules interact, how data flows between layers, and how user behavior is supported through meaningful interface design.

Since this project follows an **Agile (Iterative) Development Model**, the system is designed in incremental stages where improvements, refinements, and functional additions were made continuously after each sprint.

The system design ensures that:

- a. The software is modular and easy to maintain.
- b. Components are reusable and independently testable.
- c. The system supports scalability for future enhancements.
- d. UI/UX remains simple, intuitive, and responsive.

7.1 Primary Design Phase

The Primary Design Phase focuses on defining the high-level architecture, outlining system modules, identifying user roles, and establishing the main development blueprint.

7.1.1 Identification of System Users

Two primary users were identified:

- **User** – Can manage expenses, track income, set budgets, and view financial reports.
- **Admin** – Manages users, transactions, reports, and system activities.

7.1.2 Identification of System Modules

During this stage, three core modules were finalized:

- a. User Module
- b. Admin Module
- c. Expense & Budget Management Module

Each module contains sub-functionalities and backend operations.

7.1.3 High-Level Architectural Planning

Initial planning involved understanding:

- a. Structure of Streamlit frontend components
- b. Navigation flow between dashboard sections
- c. Backend logic using Python
- d. Database tables required: Users, Transactions, Budgets

A general hierarchy was established:

- a. UI Layer → Business Logic → Database Layer

7.1.4 Database Schema Planning

Early schema planning ensured smooth development:

- a. **Users Table** stores login information, encrypted passwords, and user roles.
- b. **Transactions Table** stores expense/income details, categories, amount, and dates.
- c. **Budgets Table** handles monthly/weekly budget information and spending limits.

7.1.5 Wireframe & Screen Flow Planning

Before coding, a rough interface blueprint was created:

- a. Landing Page
- b. Login & Signup Page
- c. Dashboard Page
- d. Expense & Income Management Page
- e. Budget Tracking Page
- f. Financial Reports Page
- g. Admin Dashboard

This helped predict user movement inside the system.

7.2 Secondary Design Phase

The Secondary Design Phase focuses on converting high-level ideas into detailed technical design. This includes database structure, module interaction, and validation logic.

7.2.1 Detailed Backend Planning

Backend modules were defined for:

- a. User Authentication (Register, Login)
- b. Transaction Management (Add, Edit, Delete, View)
- c. Budget Management (Set Budget, Update Budget)
- d. Financial Analysis & Reports

Each module follows:

- e. Input validation
- f. Error handling
- g. Secure database operations
- h. Role-based access control

7.2.2 Business Logic Mapping

Detailed logic designed for:

- a. AI-based smart expense analysis
 - Suggests spending insights based on user transaction history
- b. Budget management
 - Prevents overspending beyond budget limits
- c. Admin privileges
 - Only admin can manage system-wide records and reports
- d. User actions
 - Users can only modify their own transactions and budgets

7.2.3 Component-Based Design (Streamlit)

Frontend is divided into reusable sections:

- a. Login Form
- b. Transaction Form
- c. Dashboard Widgets
- d. Budget Panels

- e. Charts & Reports
- f. Admin Dashboard Panels

Benefits:

- a. Less repeated code
- b. Better maintainability
- c. Easy to debug and update

7.2.4 Data Flow Design

Flow between frontend and backend:

1. User submits request from Streamlit interface.
2. Python backend validates the request.
3. Business logic processes the data.
4. Database returns required information.
5. Streamlit updates UI accordingly.

7.2.5 Error & Exception Handling Design

- a. Invalid login attempts → error message
- b. Invalid transaction entries → blocked
- c. Missing form fields → validation messages
- d. Incorrect financial calculations → handled automatically

7.2.6 Security Design

- a. Password hashing
- b. Protected admin access
- c. Input sanitization and validation
- d. Secure database interaction

7.3 User Interface Design (UI/UX)

The UI/UX design ensures the system is intuitive, simple, and responsive.

7.3.1 Design Principles Used

- a. Simplicity: Minimal visual clutter

- b. Consistency: Uniform buttons, charts, and layouts
- c. Responsiveness: Works on mobile, tablet, and desktop
- d. Accessibility: Clean typography and proper spacing
- e. User-Centered Flow: Easy navigation for users and admin

7.3.2 User-Side Screens Designed

- a. Home Page – Introduction and navigation
- b. Signup Page – Register new account
- c. Login Page – Secure authentication
- d. Dashboard Page – Financial overview
- e. Expense Management Page – Add/manage expenses and income
- f. Budget Tracking Page – Set and monitor budgets
- g. Reports Page – View financial charts and analysis
- h. User Profile Page – Manage account information

7.3.3 Admin-Side Screens Designed

- a. Admin Login Page
- b. Admin Dashboard (Statistics Overview)
- c. Manage Users Page
- d. Transaction Monitoring Page
- e. Financial Reports Page
- f. System Activity Monitoring Page

7.3.4 UI Technology Stack

- a. Streamlit (Python-based frontend framework)
- b. Python
- c. HTML/CSS customization (if required)
- d. MySQL / SQLite Database

7.3.5 Overall UI Flow

- a. Users → Login → Add Transactions → Track Expenses → View Reports → Manage Budget
- b. Admin → Login → Monitor Users → Manage Trans. → View Analytics → Maintain system

CHAPTER – 8

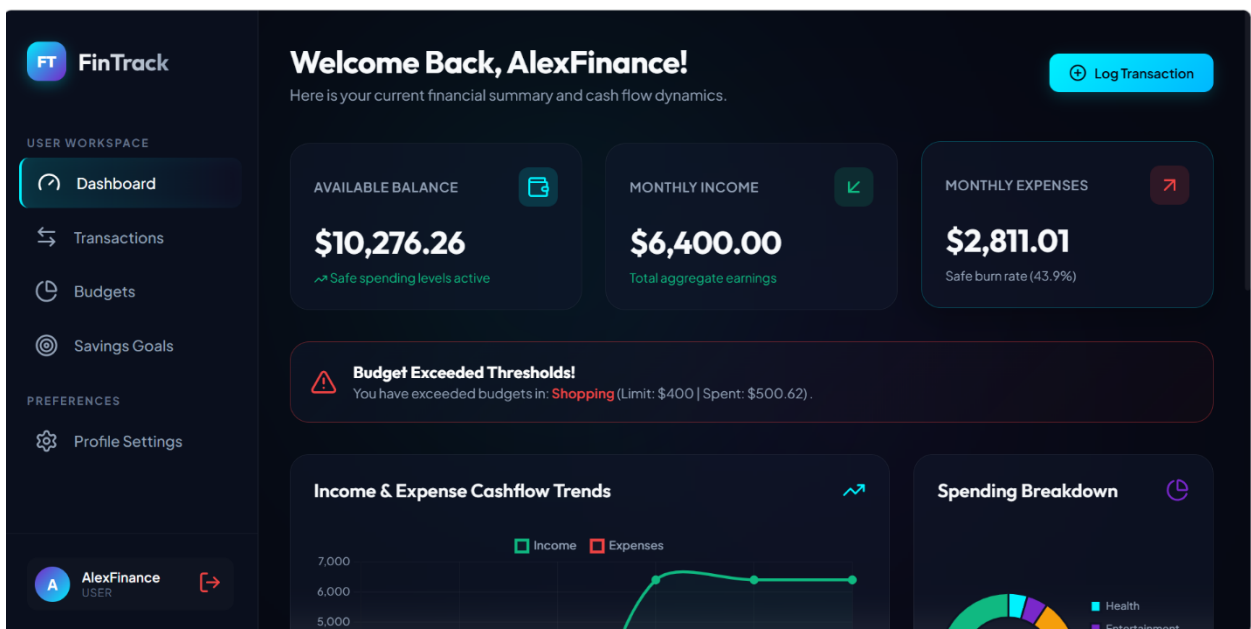
PROJECT OUTCOMES

The successful implementation of the **FinTrack – AI Powered Expense Tracker** will lead to the following outcomes:

8.1 User Module → Registration, Login, Transaction Log, Expense Tracking, Reports

a. Seamless Financial Management Platform

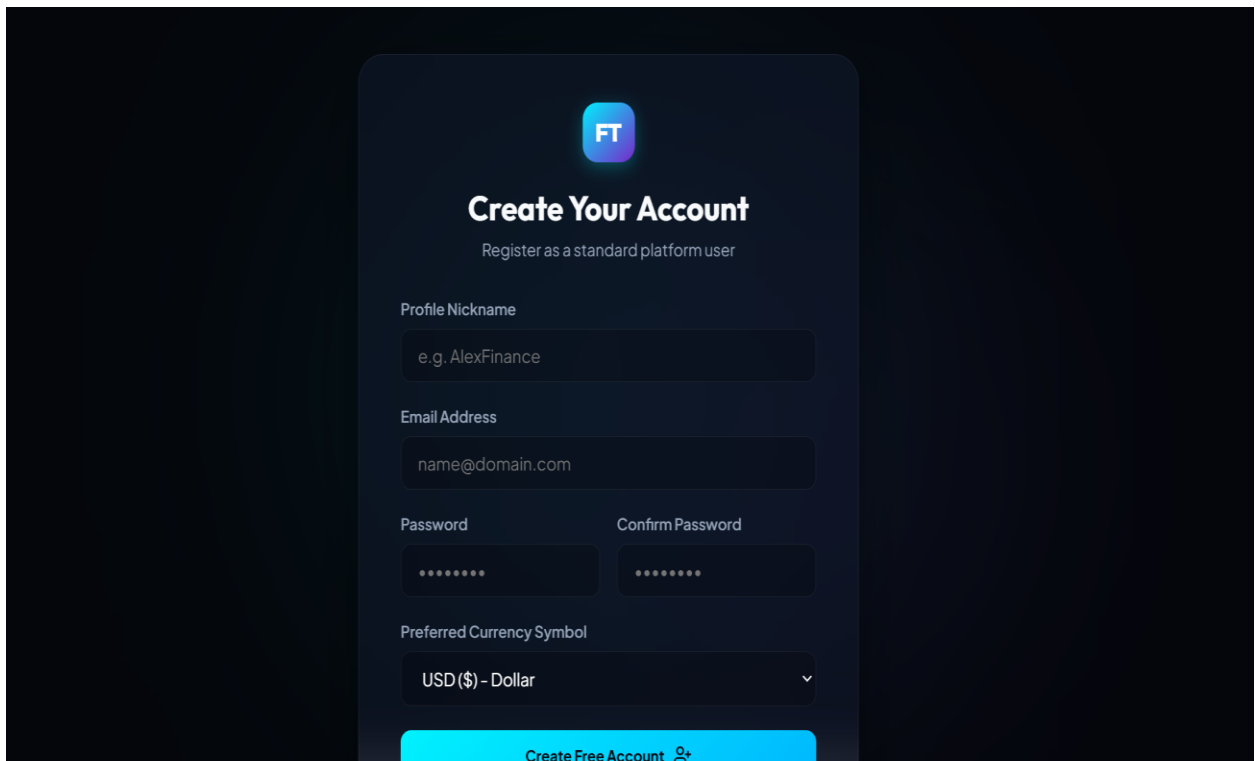
Users will be able to register, log in securely, add income and expense records, and monitor their financial activities in real time. The system provides a smooth and transparent expense management experience with accurate financial tracking



(fig. 8.1.1)

b. Secure Registration & Login System

The Login and Registration Pages act as the entry point of the FinTrack system for users and administrators. These pages allow users to securely access the system using registered email and password credentials. Proper input validation ensures that incorrect or incomplete credentials are not accepted

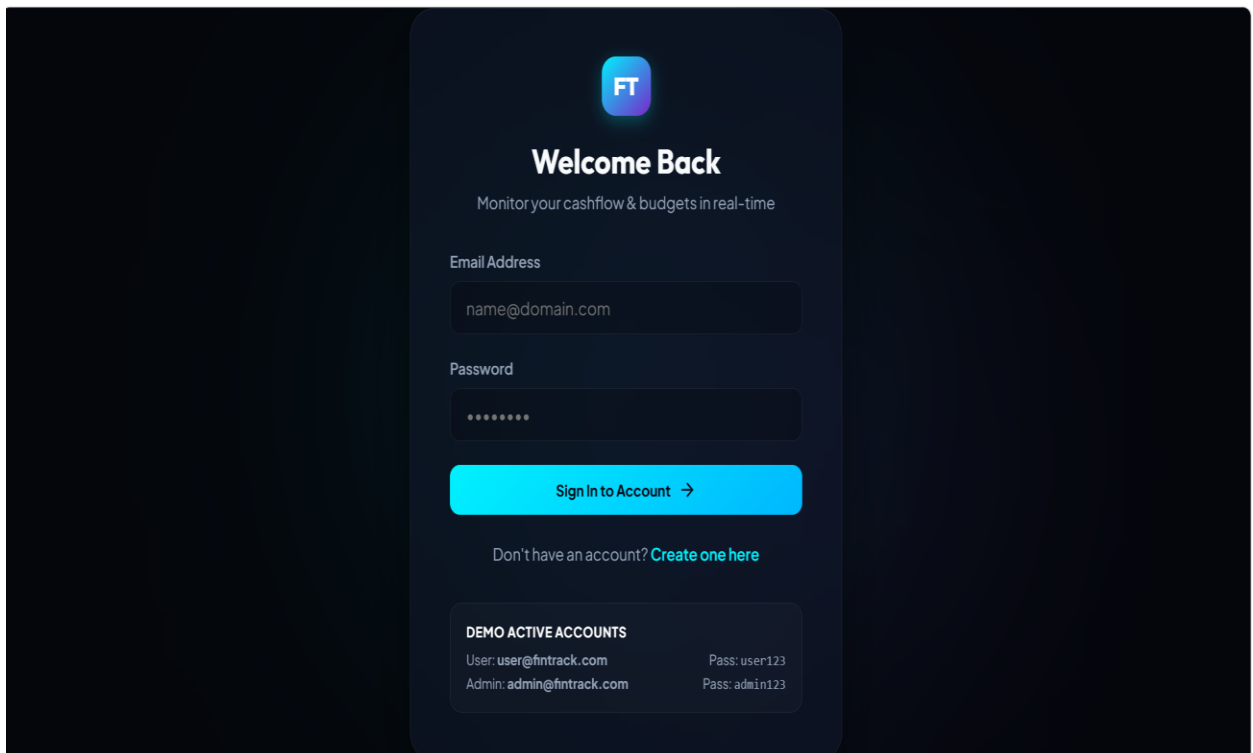


The image shows a dark-themed registration form titled "Create Your Account" with the subtitle "Register as a standard platform user". At the top is a logo with the letters "FT" in a blue and purple gradient. The form contains the following fields:

- Profile Nickname:** A text input field with the placeholder text "e.g. AlexFinance".
- Email Address:** A text input field with the placeholder text "name@domain.com".
- Password:** A text input field with masked characters (dots).
- Confirm Password:** A text input field with masked characters (dots).
- Preferred Currency Symbol:** A dropdown menu currently showing "USD (\$) - Dollar".

At the bottom of the form is a large blue button labeled "Create Free Account" with a small icon of a person with a plus sign.

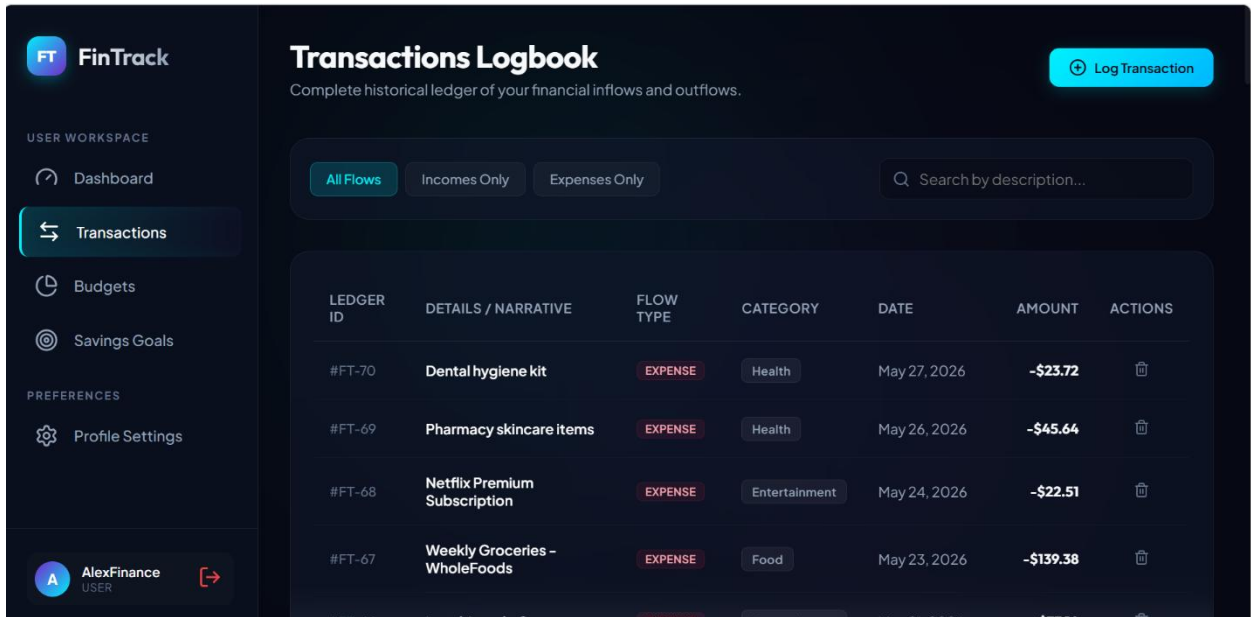
(Fig. 8.1.2)



(Fig. 8.1.3)

c. Efficient Transaction Management

Users can easily add, edit, and delete financial transactions. The Transaction Log maintains records of expenses and income, helping users track their spending habits effectively



(Fig. 8.1.4)

d. Expense Analysis & Report Generation

The Report Page provides detailed financial summaries, expense analysis, and graphical reports. Users can view category-wise spending, monthly expense reports, and overall financial statistics for better money management



(Fig. 8.1.5)

f. Enhanced User Experience

The system provides a clean dashboard with real-time updates, responsive navigation, and easy-to-understand charts, improving overall user engagement and financial awareness.

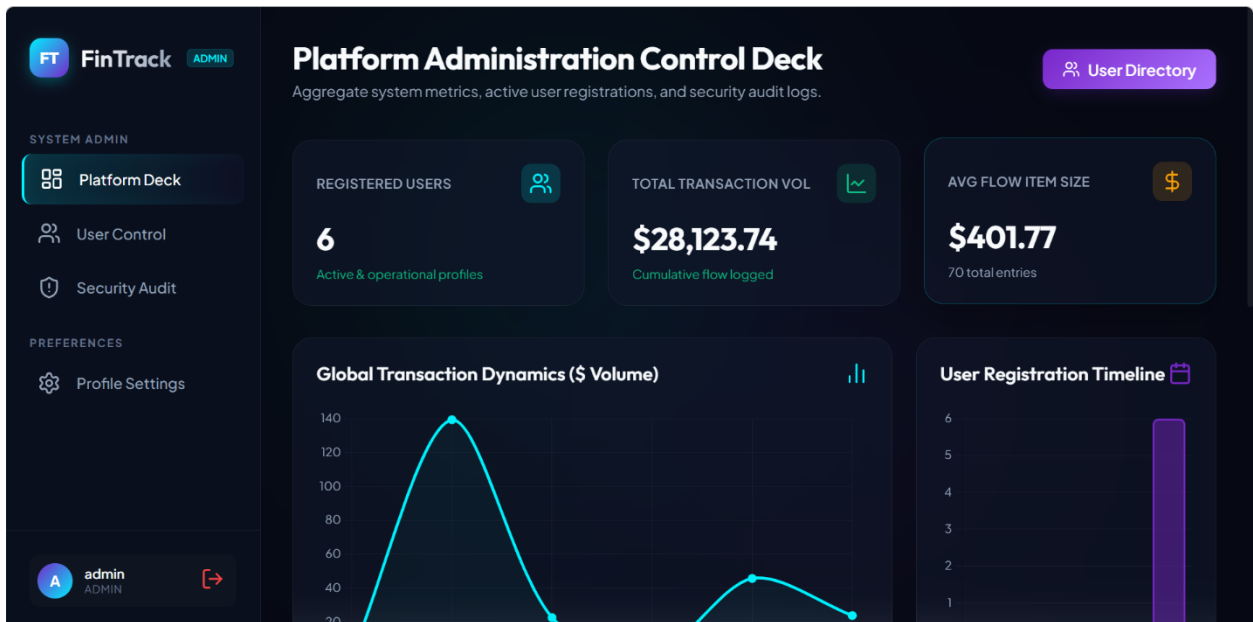
g. Automation of Financial Tracking

Manual financial calculations and paper-based expense tracking are replaced with an automated digital solution. This reduces calculation errors, saves time, and improves efficiency in managing personal finances.

8.2 Admin Module → Dashboard, User Management, Transaction Monitoring

a. Admin Dashboard Management

The Admin Module is a core component of the FinTrack system designed to provide complete monitoring and management functionality to the administrator. The dashboard allows the admin to monitor users, transactions, and overall system activities efficiently.



(Fig. 8.2.1)

b. User and Transaction Monitoring

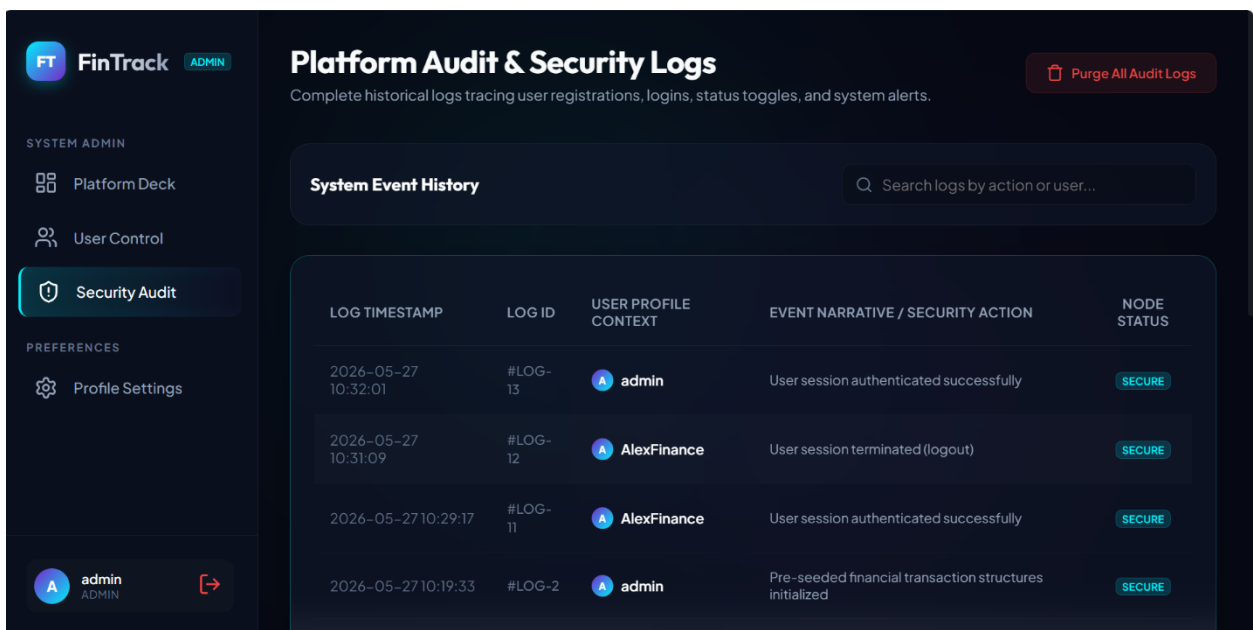
The admin can manage registered users, monitor transaction records, and remove invalid or suspicious entries from the system. This ensures that financial data remains organized, secure, and updated .



(Fig. 8.2.2)

c. Financial Reports & Analytics

The admin dashboard provides summaries of total users, total transactions, and financial statistics. This functionality helps monitor system performance and maintain smooth operation of the expense tracking platform .



(Fig. 8.2.3)

Contribution to Digital Transformation

The project demonstrates how modern Python technologies and Streamlit-based applications can transform traditional financial management methods into smart digital solutions, promoting financial awareness and efficient expense tracking.

8.3 Database Module

a. Transaction Database Management

The Transactions table maintains financial records including user ID, transaction category, amount, date, transaction type, and balance information

b. User Database Management

The Users table contains user-related data such as username, email, encrypted password, phone number, and account details. This ensures secure user authentication and efficient account manage

CHAPTER – 9

TESTING OF FINTRACK – AI POWERED FINANCE TRACKER

Software testing is one of the most critical phases in the development life cycle. It ensures that the system performs as expected, delivers accurate results, and functions consistently across all usage scenarios. The **FinTrack – AI Powered Expense Tracker** was tested comprehensively using Manual Testing, with a focus on validating the correctness of each module, identifying functional issues, ensuring smooth user experience, and verifying database integrity.

9.1 Introduction to Testing

Testing is an essential process carried out to verify that the software system meets all the defined requirements and performs reliably in real-world conditions. It ensures the detection of defects, inconsistencies, incorrect logic, performance issues, and failures before the system is deployed. For the **FinTrack – AI Powered Expense Tracker**, testing was conducted with the following objectives:

- To ensure that user activities such as registration, login, adding transactions, viewing expense reports, and managing financial records are functioning correctly.
- To validate admin operations including monitoring users, viewing transactions, and managing reports.
- To confirm that financial calculations such as total expenses, income summaries, and balance tracking operate accurately.
- To verify that all transaction and user data is stored correctly in the database.
- To check responsiveness, usability, and compatibility across multiple browsers and devices.

The primary goal of testing was to deliver a robust, stable, and user-friendly financial management application suitable for real-world usage.

9.2 Types of Testing Conducted

Multiple testing methods were applied to validate all modules of the **FinTrack – AI Powered Expense Tracker**. Each testing type contributed to improving system reliability and performance.

1. Functional Testing

Functional Testing ensures every feature of the application works exactly as intended according to the requirement specification.

Key Areas Tested:

- User registration and authentication
- Admin login and access verification
- Expense and income entry operations
- Transaction log display and filtering
- Financial report generation
- Admin operations for monitoring users and transactions
- Viewing and updating database records

2. Usability Testing

Usability Testing focuses on the ease of use, intuitiveness, and smoothness of the interface.

Tasks Observed:

- Whether a new user can use the expense tracker without assistance
- Clarity of dashboard navigation and menus
- Readability of text, labels, charts, and instructions
- Responsiveness on different screen sizes

3. Validation Testing

Validation Testing ensures that the system accepts only valid inputs and rejects invalid entries.

Examples:

- Preventing duplicate email registration
- Restricting login with incorrect credentials
- Preventing invalid transaction entries
- Ensuring expense amount fields accept only numeric values

4. Compatibility Testing

Compatibility Testing validates how the system performs across different browsers and devices.

Browsers Tested:

- a. Google Chrome
- b. Mozilla Firefox

The UI was checked for consistency in:

- Layout
- Colors
- Fonts
- Alignment
- Button responsiveness

5. Database Testing

Database Testing ensures that all CRUD (Create, Read, Update, Delete) operations are correctly performed in the database.

Database Tests Included:

- Insertion of new users
- Storing expense and income records
- Updating transaction details
- Deleting transaction records
- Fetching latest financial information

Database accuracy is essential because reports, balances, and financial summaries depend on real-time data.

9.3 Test Plan and Execution

The test plan defines the scope, objectives, resources, responsibilities, and activities involved in the testing process.

Testing Objectives

- a. Ensure accuracy in user and admin modules
- b. Verify financial calculations and transaction records
- c. Validate data flow between frontend, backend, and database

- d. Identify usability and navigation issues

Testing Strategy

- a. Manual execution of all test scenarios
- b. Use of real-time test data including valid, invalid, and edge cases
- c. Browser testing on Chrome and Firefox
- d. Database validation using MySQL/SQLite database tools
- e. Inspection of logs and error handling during execution

Test Environment

- Frontend: Streamlit
- Backend: Python
- Database: MySQL / SQLite
- OS Used: Windows 10
- Browsers: Chrome & Firefox

Execution Approach

Testing was carried out in phases:

Phase 1: Module Testing

Each module—Registration, Login, Transactions, Reports, and Admin—was tested independently.

Phase 2: Integration Testing

Combined modules were tested together:

- a. Login → Dashboard → Add Transaction → View Reports

Phase 3: System Testing

Complete system workflow tested from end-to-end.

Phase 4: Regression Testing

Modules were re-tested after fixing detected bugs and issues.

This systematic testing approach ensured high reliability and software quality.

9.4 Challenges Faced

During the testing phase, several challenges were encountered:

1. Managing Real-Time Financial Updates

Ensuring instant updates in expense summaries and dashboard reports required multiple rounds of validation and optimization.

2. Streamlit UI Refresh Handling

Certain dashboard components did not refresh immediately after transaction updates. This issue was resolved by improving session state management in Streamlit.

3. Browser Compatibility Adjustments

Some UI elements appeared differently in Firefox compared to Chrome, requiring layout and styling refinements.

4. Database Synchronization Delays

At times, updated transaction records took a few seconds to reflect in reports, requiring repeated database testing and query optimization.

5. Error Handling Improvements

Clear and user-friendly validation messages had to be implemented for all scenarios such as invalid login attempts, incorrect transaction amounts, and incomplete form submissions.

CHAPTER – 10

PROPOSED TIME DURATION

The development of the **FinTrack – AI Powered Expense Tracker** is planned to be completed within **7–8 weeks**, following a systematic and structured timeline. The tentative phase-wise duration is as follows:

1. Phase 1 – Requirement Analysis & Planning (1–2 Weeks)

Understanding the project requirements and financial management workflow.

- Gathering requirements related to expense tracking, transaction management, and report generation.
- Finalizing project modules, technology stack, and database structure.
- Preparing the project roadmap and development timeline.
- Identifying user and admin functionalities.

2. Phase 2 – Frontend Development (2 Weeks)

- Designing responsive user interfaces using Streamlit.
- Developing modules such as Login, Signup, Dashboard, Transaction Log, and Report Pages.
- Creating reusable UI sections and dashboard components for better maintainability.
- Ensuring responsive design for desktop, mobile, and tablet devices.

3. Phase 3 – Backend Development (2–3 Weeks)

- Developing backend logic using Python.
- Implementing functionalities for user authentication and transaction management.
- Creating modules for expense tracking, budget calculation, and report generation.
- Implementing security features such as password hashing, session management, and error

handling.

4. Phase 4 – Database Integration (1 Week)

- Designing and integrating MySQL / SQLite database tables.
- Managing user data, expense records, income details, and transaction history.
- Enabling real-time financial updates and efficient database queries.

5. Phase 5 – Testing & Debugging (1 Week)

- Conducting unit testing, integration testing, and functional testing.
- Fixing bugs and optimizing system performance.
- Performing usability, validation, and security testing.
- Verifying report accuracy and financial calculations.

6. Phase 6 – Maintenance & Updates (Ongoing)

- Monitoring application performance and database operations.
- Addressing post-deployment issues and fixing errors.
- Rolling out updates and preparing future enhancements such as AI-based expense analysis, cloud integration, and advanced financial analytics.

CONCLUSION

The **FinTrack – AI Powered Expense Tracker** developed as part of this major project demonstrates the successful implementation of modern Python-based technologies including Streamlit, Python, and MySQL/SQLite to build an intelligent, responsive, and scalable financial management system. The primary objective of this project was to simplify and modernize traditional expense tracking methods by providing a centralized digital platform where users can efficiently manage income, expenses, transaction records, and financial reports from anywhere.

The project was initiated after identifying major challenges in traditional financial management systems such as manual record maintenance, calculation errors, lack of organized expense tracking, limited accessibility, and difficulty in analyzing spending patterns. Conventional methods of maintaining financial records using notebooks or spreadsheets often fail to provide real-time financial insights and accurate expense management. The developed FinTrack platform successfully addresses these challenges by digitizing the complete financial workflow and automating major operations including user registration, authentication, transaction management, expense tracking, report generation, and dashboard analysis.

The application follows a structured architecture where Streamlit is used for building a dynamic, interactive, and responsive user interface. The frontend design ensures smooth navigation, fast rendering, and compatibility across desktops, laptops, tablets, and mobile devices. The backend, developed using Python, efficiently manages business logic, authentication, database communication, financial calculations, and report generation. MySQL/SQLite is used as the database system to securely store user information, expense records, income details, transaction history, and financial reports.

The system architecture follows modular development principles that improve maintainability, scalability, and code reusability. The project includes multiple modules such as authentication, dashboard management, transaction handling, expense tracking, report generation, and admin management. The responsive dashboard allows users to add and manage financial transactions while monitoring their income, expenses, savings, and budgets effectively.

The project was tested thoroughly using manual testing methodologies. Major functionalities such as signup, login, transaction entry, expense calculations, report generation, dashboard rendering,

responsive design, database operations, and browser compatibility were validated successfully. Testing confirmed that the system performs accurately and remains stable under different usage scenarios. Compatibility testing across modern browsers such as Google Chrome and Mozilla Firefox ensured consistent user interface behavior and proper responsiveness.

From a technical and practical perspective, this project provided valuable exposure to Python-based application development, Streamlit dashboard development, database integration, financial data processing, authentication systems, responsive interface design, and scalable architecture planning. The knowledge and experience gained during the development process align with current industry standards and strengthen practical software development skills.

In conclusion, the **FinTrack – AI Powered Expense Tracker** successfully fulfills its objectives by providing a secure, efficient, user-friendly, and scalable financial management solution. The project demonstrates how modern technologies can transform traditional expense tracking methods into a smart digital system that improves financial awareness, accuracy, and productivity for users.

REFERENCES

1. Grinberg, M. *Flask Web Development*. O'Reilly Media, 2018.
2. Ramalho, L. *Fluent Python*. O'Reilly Media, 2022.
3. McKinney, W. *Python for Data Analysis*. O'Reilly Media, 2022.
4. Matthes, E. *Python Crash Course*. No Starch Press, 2023.
5. Downey, A. *Think Python*. O'Reilly Media, 2015.
6. Summerfield, M. *Programming in Python 3*. Addison-Wesley, 2010.
7. Beazley, D. *Python Essential Reference*. Addison-Wesley, 2009.
8. Guttag, J. *Introduction to Computation and Programming Using Python*. MIT Press, 2021.
9. Lutz, M. *Learning Python*. O'Reilly Media, 2013.
10. Al Sweigart. *Automate the Boring Stuff with Python*. No Starch Press, 2020.
11. VanderPlas, J. *Python Data Science Handbook*. O'Reilly Media, 2016.
12. Nelli, F. *Python Data Analytics*. Apress, 2018.
13. Pilgrim, M. *Dive Into Python 3*. Apress, 2009.
14. Slatkin, B. *Effective Python*. Addison-Wesley, 2019.
15. Reitz, K. *The Hitchhiker's Guide to Python*. O'Reilly Media, 2016.
16. Streamlit Documentation. Available at: [Streamlit Official Website](#)
17. Python Software Foundation. Available at: [Python Official Website](#)
18. SQLite Documentation. Available at: [SQLite Official Website](#)
19. MySQL Documentation. Available at: [MySQL Official Website](#)
20. GitHub Documentation. Available at: [GitHub Official Website](#)
21. Visual Studio Code Documentation. Available at: [VS Code Official Website](#)
22. Mozilla Developer Network. Available at: [MDN Web Docs](#)
23. W3Schools Online Web Tutorials. Available at: [W3Schools](#)
24. Sommerville, I. *Software Engineering*. Pearson Education, 2016.
25. Pressman, R. *Software Engineering: A Practitioner's Approach*. McGraw-Hill, 2014.
26. Jalote, P. *An Integrated Approach to Software Engineering*. Springer, 2005.
27. Roger, S. *Principles of Information Systems*. Cengage Learning, 2018.
28. Elmasri, R., and Navathe, S. *Fundamentals of Database Systems*. Pearson, 2017.

29. Coronel, C. *Database Systems: Design, Implementation, and Management*. Cengage Learning, 2019.
30. Silberschatz, A., Korth, H., and Sudarshan, S. *Database System Concepts*. McGraw-Hill, 2019.
31. Date, C.J. *An Introduction to Database Systems*. Pearson, 2003.
32. Gamma, E. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
33. Martin, R.C. *Clean Code*. Prentice Hall, 2008.
34. Freeman, E. *Head First Design Patterns*. O'Reilly Media, 2020.
35. Nielsen, J. *Usability Engineering*. Morgan Kaufmann, 1994.
36. Krug, S. *Don't Make Me Think*. New Riders, 2014.
37. Shneiderman, B. *Designing the User Interface*. Pearson, 2016.
38. Stallings, W. *Computer Security Principles and Practice*. Pearson, 2018.
39. Bishop, M. *Computer Security: Art and Science*. Addison-Wesley, 2018.
40. OWASP Foundation. Available at: [OWASP Official Website](#)
41. Mozilla Firefox Documentation. Available at: [Mozilla Firefox Official Website](#)
42. Google Chrome Documentation. Available at: [Google Chrome Official Website](#)
43. Agile Alliance. Available at: [Agile Alliance Official Website](#)
44. IEEE Software Engineering Standards. Available at: [IEEE Official Website](#)
45. Oracle Documentation for SQL. Available at: [Oracle Official Website](#)
46. TutorialsPoint Python Tutorials. Available at: TutorialsPoint
47. GeeksforGeeks Python Tutorials. Available at: [GeeksforGeeks](#)
48. Real Python Tutorials. Available at: [Real Python](#)
49. Kaggle Data Science Tutorials. Available at: [Kaggle Official Website](#)
50. Stack Overflow Developer Community. Available at: [Stack Overflow Official Website](#)